What is Quality?

Quality is meeting the requirement, expectation, and needs of the customer is free from the defects, lacks and substantial variants. There are standards needs to follow to satisfy the customer requirements.

What is software quality?

The quality of software can be defined as the ability of the software to function as per user requirement. When it comes to software products it must satisfy all the functionalities written down in the software requirement specification(SRS)document.

Key aspects that conclude software quality include,

- **Good design** – It's always important to have a good and aesthetic design to please users
- **Reliability** – Be it any software it should be able to perform the functionality impeccably without issues
- **Durability-** Durability is a confusing term, In this context, durability means the ability of the software to work without any issue for a long period of time.
- **Consistency** – Software should be able to perform consistently over platform and devices
- **Maintainability** – Bugs associated with any software should be able to capture and fix quickly and news tasks and enhancement must be added without any trouble
- **Value for money –** customer and companies who make this app should feel that the money spent on this app has not fone to waste.

What are software quality metrics?

Metrics are pointers or numbers which help you understand the attributes of a **p**roduct, (like its complexity, its size, it's quality, etc.), the attributes of the **p**rocess and the attributes of the **p**roject (which includes the number of resources, costs, productivity and timeline among others), popularly known as the three P's.

Important Software Quality Metrics

1. Defect Density

The first measure of the quality of any products is the number of defects found and fixed. The more the number of defects found, would be the quality of development is poor. So the management should strive hard to improve development and do an RCA

(Root Cause Analysis) to find why the quality is taking the hit.

Defect Density = No. of Defects Found / Size of AUT or module

2. **Defect Removal Efficiency (DRE)**

This is an important metric for assessing the effectiveness of a testing team. DRE is an indicator of the number of defects the tester or the testing team was able to remove from going into a production environment. Every quality team wants to ensure a 100% DRE.

DRE = A/(A+B) x 100

A – number of defects found before production

B – Number of defects found in production

3. **Meantime between failures (MTBF) (imp)**

As the name suggests it is the average time between two failures in a system. Based on the AUT and expectation of business the definition of failure may vary.

For any online website or mobile application crash or disconnection with the database could be the expected failure. No team can produce software that never breaks or fails, so the onus is always to increase the MTBF as much as possible, which means that in a time frame the number of times the applications fail should be reduced to an acceptable number.

4. **Meantime to recover (MTTR) (imp)**

This again is quite self-explanatory. The mean time to recover is basically the time it takes for the developers to find a critical issue with the system, fix it and push the fix patch to production. Hence the average time which the team needs to fix an issue in production. It is more of maintenance contract metrics, where an MTTR of 24 hours would be preferred over an MTTR of 2 days for obvious reasons.

5. **Application Crash Rate**

Important metrics especially for mobile apps and online websites. It is a measure of how often the mobile app or website crashes in any environment. It is an indicator of the quality of the code. The better the code, the longer it will be able to sustain without crashing.

7. **Cycle Time**

Cycle time is similar to the lead time with a difference that leads time is measured per user story, while cycle time is measured per task. For eg, if database creation is part of the user story related to client data.

8. **Team Velocity**

Team Velocity is a very important metric for Agile/Scrum. It is an indicator of the number of tasks or user stories a team is able to complete during a single sprint.

Cost of Quality :
It is the most established, effective measure of quantifying and calculating the business value of testing. There are four categories to measure cost of quality: Prevention costs, Detection costs, Internal failure costs, and External failure costs. These are explained as follows below.

1. **Prevention costs** include cost of training developers on writing secure and easily maintainable code
2. **Detection costs** include the cost of creating test cases, setting up testing environments, revisiting testing requirements.
3. **Internal failure costs** include costs incurred in fixing defects just before delivery.
4. **External failure costs** include product support costs incurred by delivering poor quality software.

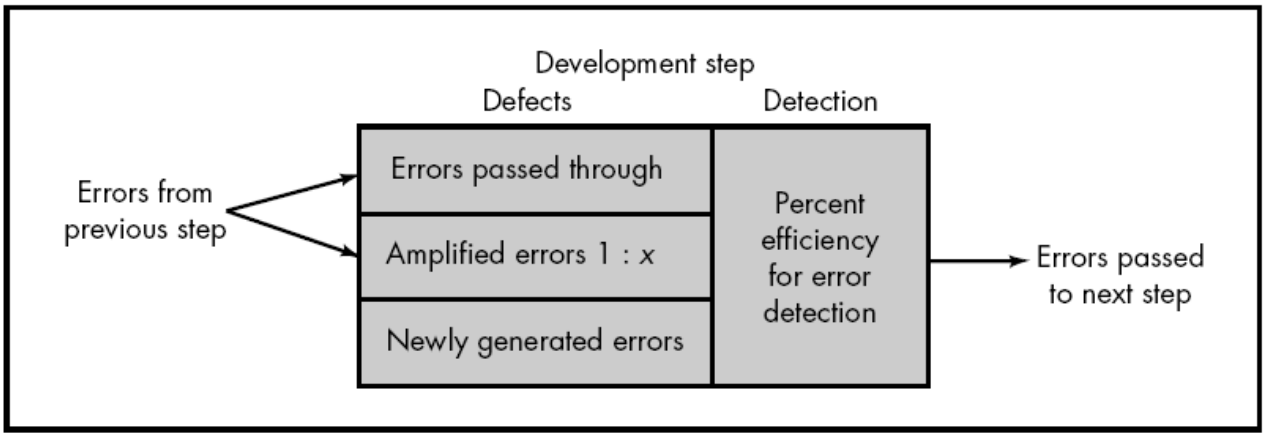
Cost of defects:

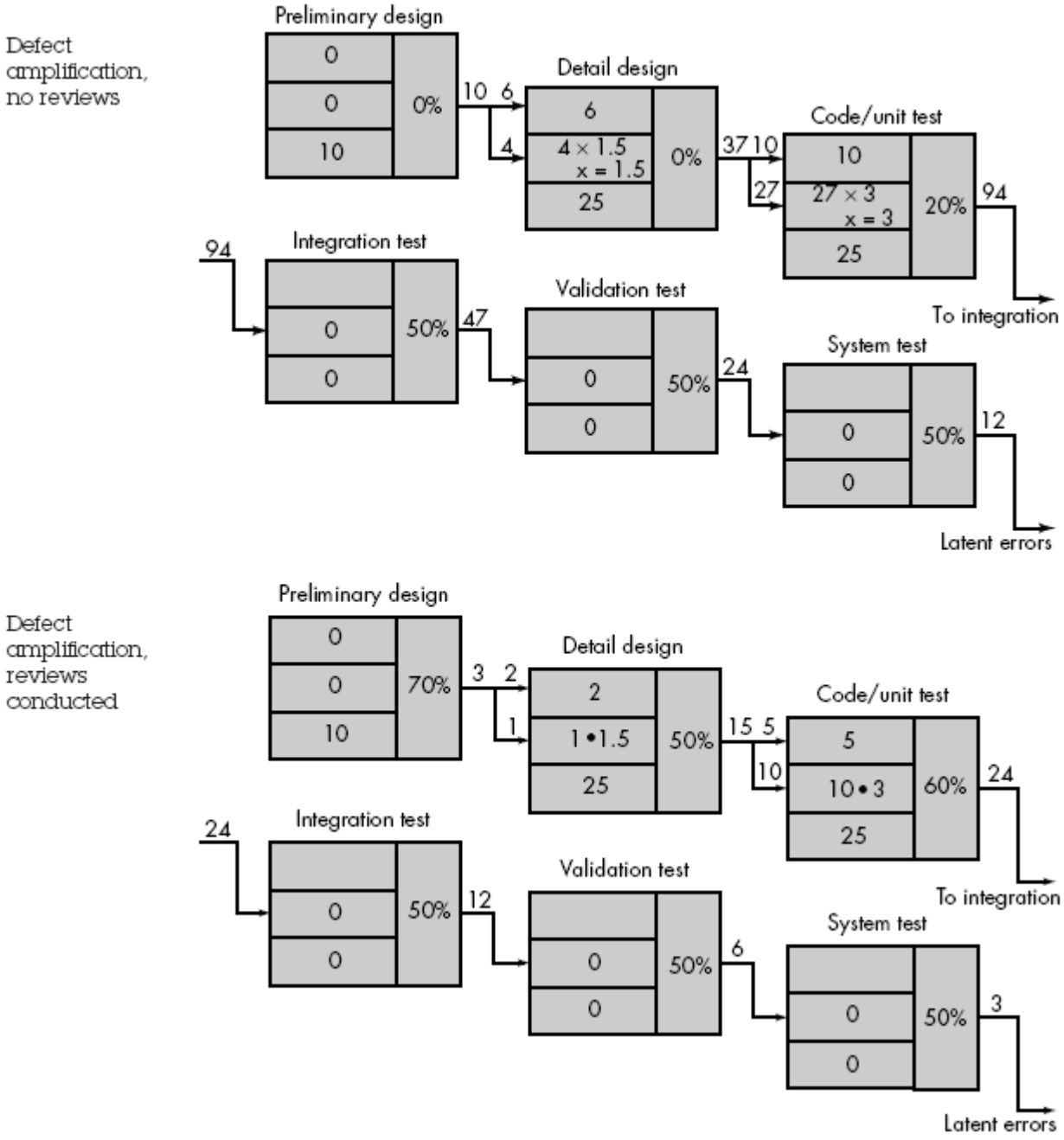
The cost of **defects** can be measured by the impact of the defects and **when we find them**. Earlier the defect is found lesser is the cost of defect. For example if error is found in the requirement specifications during requirements gathering

and analysis, then it is somewhat cheap to fix it. The correction to the requirement specification can be done and then it can be re-issued.

Defect Amplification and Removal

A defect amplification model [IBM81] can be used to illustrate the generation and detection of errors during the preliminary design, detail design, and coding steps of the software engineering process. A box represents a software development step.





What is Assurance?

Assurance is provided by organization management, it means giving a positive declaration on a product which obtains confidence for the outcome. It gives a security that the product will work without any glitches as per the expectations or requests.

What is Quality Assurance?

Quality Assurance is known as QA and focuses on preventing defect. Quality Assurance ensures that the approaches, techniques, methods and processes are designed for the projects are implemented correctly. Quality assurance activities monitor and verify that the processes used to manage and create the deliverables have been followed and are operative. Quality Assurance is a proactive process and is Prevention in nature. It recognizes flaws in the process. Quality Assurance has to complete before Quality Control.

what is Control?

Control is to test or verify actual results by comparing it with the defined standards.

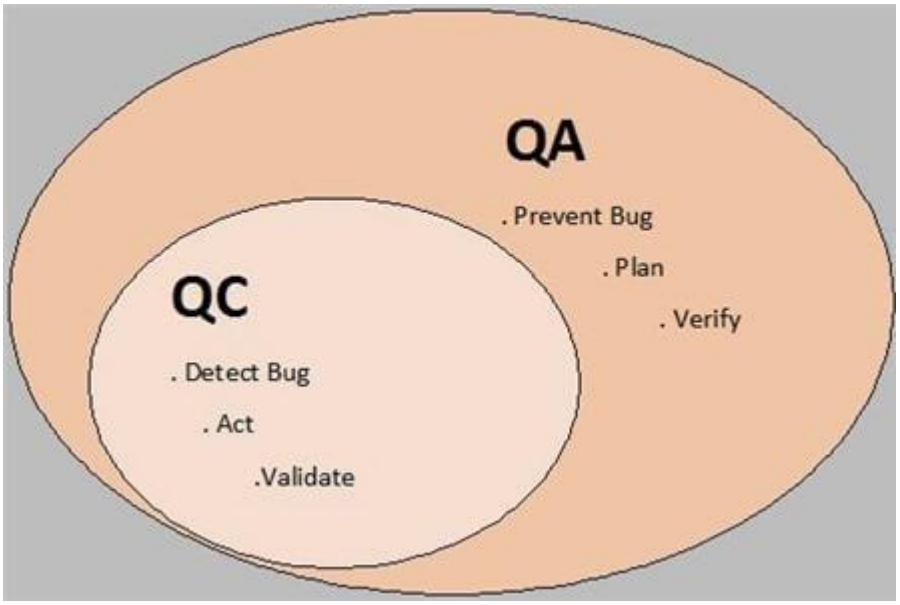
What is Quality Control?

Quality Control is known as QC and focuses on identifying a defect. QC ensures that the approaches, techniques, methods and processes are designed in the project are following correctly. QC activities monitor and verify that the project deliverables meet the defined quality standards.

Quality Control is a reactive process and is detection in nature. It recognizes the defects. Quality Control has to complete after Quality Assurance.

Quality Assurance	**Quality Control**
It is a process which deliberates on providing assurance that quality request will be achieved.	QC is a process which deliberates on fulfilling the quality request.
A QA aim is to prevent the defect.	A QC aim is to identify and improve the defects.
QA is the technique of managing quality.	QC is a method to verify quality.
QA does not involve executing the program.	QC always involves executing the program.

Quality Assurance	Quality Control
All team members are responsible for QA.	Testing team is responsible for QC.
QA Example: Verification	QC Example: Validation.
QA means Planning for doing a process.	QC Means Action for executing the planned process.
Statistical Technique used on QA is known as Statistical Process Control (SPC.)	Statistical Technique used on QC is known as Statistical Quality Control (SPC.)



Software review: (imp)

A software review is an effective way of filtering errors in a software product. Reviews conducted at each of these phases i.e., analysis, design, coding, and testing reveal areas of improvement in the product. Reviews also indicate those areas that do not need any improvement. Reviews also make the task of product creation more manageable. Some of the most common software review techniques are:
i. Inspection
ii. Walkthrough
iii. Code review
iv. Formal Technical Reviews (FTR)
v. Pair programming

Formal technical review (FTR): (imp)

1. A formal technical review is a software quality assurance activity performed by software engineers.
2. In addition, the FTR serves as a training ground, enabling junior engineers to observe the different approaches to software analysis, design, and implementation.
3. The objectives of FTR are
   i. To uncover errors in function, logic, or implementation for representation of software.
   ii. To verify that software under review meets its requirements.
   iii. To ensure that the software has been represented according to predefined standards.
   iv. To achieve software that is developed in a uniform manner. v. To make projects more manageable.

Steps required to conduct a successful FTR:

1. The review meeting

- Every review meeting should be conducted by considering the following constraints:
  a. Short duration Advance preparation
  b. Involvement of people
  c. walkthrough
- Rather than attempting to review the entire design; is conducted for modules or for a small group of modules.
- The focus of the FTR is on the work product (a software component to be review). The review meeting is attend by the review leader., all reviewers, and the producer.
- One of the reviewers becomes a recorder who records all the important issues raised during the review. When errors are discovere, the recorder notes each error.
- At the end of the review, the attendees decide whether to accept the product or not, with or without modification.

2. Review reporting and record-keeping

- During the FTR, the reviewer actively records all the issues that have been raised.

- At the end of the meeting, these all raised issues are consolidating and a review issues list is prepare.
  Finally, a formal technical review summary report is produce.

3. Review guidelines

- Guidelines for the conducting of formal technical review must be establish in advance.
- These guidelines must be distribute to all reviewers, agree upon, and then followed.

Software Quality Assurance (imp)

Software quality assurance (or SQA for short) is the ongoing process that ensures the software product meets and complies with the organization's established and standardized quality specifications. SQA is a set of activities that verifies that everyone involved with the project has correctly implemented all procedures and processes. SQA's ultimate goal is to catch a product's shortcomings and deficiencies before the general public sees it. If mistakes get caught in-house, it means fewer headaches for the development team and a lot less angry customers.

ten vital elements:

1. Software engineering standards
2. Technical reviews and audits
3. Software testing for quality control
4. Error collection and analysis
5. Change management
6. Educational programs
7. Vendor management
8. Security management
9. Safety
10. Risk management

Goal of SQA

1. Requirement quality : The correctness, completeness, and consistency of the requirements model will have a strong influence on the quality of all work products that follow. SQA must ensure that the software team has properly reviewed the requirements model to achieve a high level of quality.

2. Design quality : Every element of the design model should be assessed y the software team to ensure that it exhibits high quality and that the design itself conforms to requirements.

3.Code quality : Source code and related work products must conform to local coding standards and exhibit characteristics that will facilitate maintainability.

4.Quality control effectiveness : A software team should apply limited resources in a way that has the highest likelihood of achieving a high–quality result. SQA analyzes the allocation of resources for reviews and testing to assess whether they are being allocated in the most effective manner.

Goal	**Attribute**	**Metric**
Requirement quality	Ambigully	Number of ambiguous modifiers (e.., many, large, human–friendly)
	Completeness	Number of TBA, TBD
	Understandability	Number of sections/subsections
	Volatility	Number of changes per requirement Time (by activity) when change is requested

	Traceability	Number of requirements not traceable to design/code
	Model clarity	Number of UML models Number of descriptive pages per model Number of UML errors
Design quality	Architectural integrity Component completeness Interface complexity Patterns	Existence of architectural model Number of components that trace to architectural model Complexity of procedural design Average number of pick to get to a typical function or content Layout appropriateness Number of patterns used
Code quality	Complexity Maintainability Understandability Reusability Documentation	Cyclomatic complexity Design factors (Chapter 8) Percent internal comments Variable naming conventions Percent reused components Readability index
QC effectiveness	Resource allocation Completion rate Review effectiveness Testing effectiveness	Staff hour percentage per activity Actual vs. budgeted completion time See review metrics Number of errors found and criticality Effort required to correct an error Origin of error

Software Reliability

Software reliability refers to a critical component of computer system availability, indicating whether users can expect a software program to perform consistently.Software Reliability is the probability of failure-free software operation for a specified period of time in a specified environment. Software Reliability is also an important factor affecting system reliability. The high complexity of software is the major contributing factor of Software Reliability problems.

reliability metrics

Availability (AVAIL)

AVAIL of software measures the possibility of availability of the system for use over a specified period of time. It calculates the number of failures that happen during a specific period and considers the length of downtime that results when a failure happens.

This is an important metric for software that causes major effects or damage when outages occur, such as telecommunication and operating systems.

Mean Time Between Failure (MTBF)

Mean Time to Repair (MTTR)

When software fails, it takes time to fix the error. MTTR measures the average time it takes to track the cause and repair the software fault.

Developers use this metric to understand the length of time needed to fix an error once a failure occurs. Teams look to this number to better understand their working process for reliability and find ways to improve it.

Mean Time to Failure (MTTF)

MTTF examines the amount of time between two occurrences of failure in software usage under normal operating conditions. The metric is an average of the time elapsed between two failures over the total number of failures.

Software failures are almost unavoidable, but this number attempts to quantify how long the software can operate without experiencing a failure. However, it doesn't look at the time it takes to fix the error and get the software running again.

ISO Standards:

ISO (International Organization for Standardization) is an independent, non-governmental international organization with a membership of 168 national standards bodies.

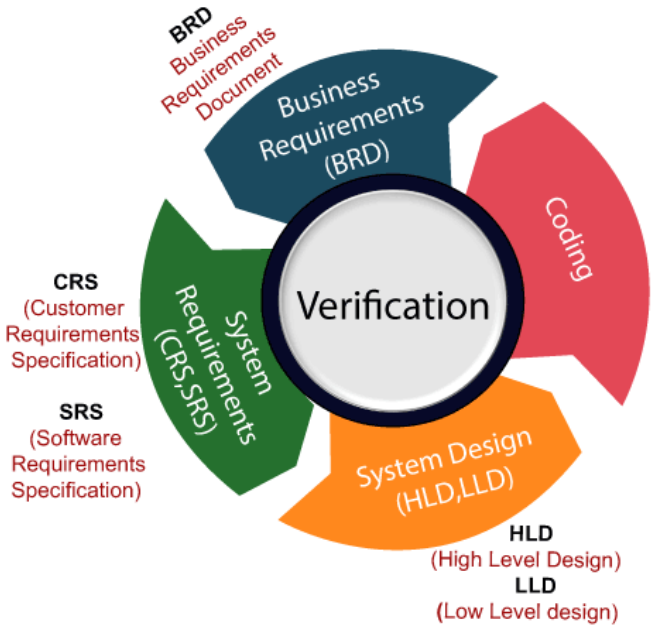
The standard having the number 29119 is developed for maintaining the correct software testing procedures for the software development. ISO/IEC/IEEE Standard 29119 is a collection of standards for software testing of any SDLC phases for any organization.

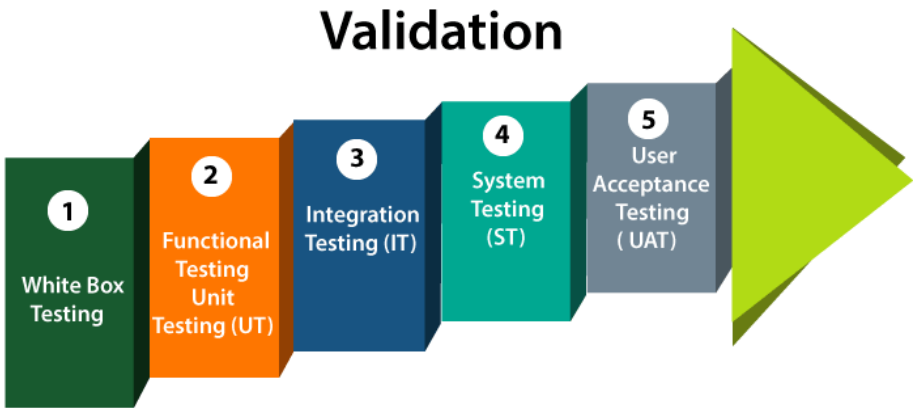
ISO/IEC/IEEE 29119 consists of five standards for international software testing standards which are:

- ISO/IEC 29119-1: This standard gives the concepts and meanings of software which are very useful in software development processes. This standard was published in September 2013.
- ISO/IEC 29119-2: This standard is also sub part of ISO Standard 29119 which deals with all the test processes for a better product output. This standard was also published in September 2013.
- ISO/IEC 29119-3: This standard has other importance related to the documentation of the product; therefore, it is responsible for delivering the complete documentation of the product. This standard is also published with previous 2 standards just discussed above, in September 2013.
- ISO/IEC 29119-4: This standard gives the right testing techniques and strategies for doing the software testing. This standard was published in 2014.
- ISO/IEC 29119-5: ISO 29119-5 has some unique importance that deals with keyword-based software testing which means that a keyword is used in complete testing and obtained the results, the keyword can be any word which can give better testing results. This Standard was published in 2015.

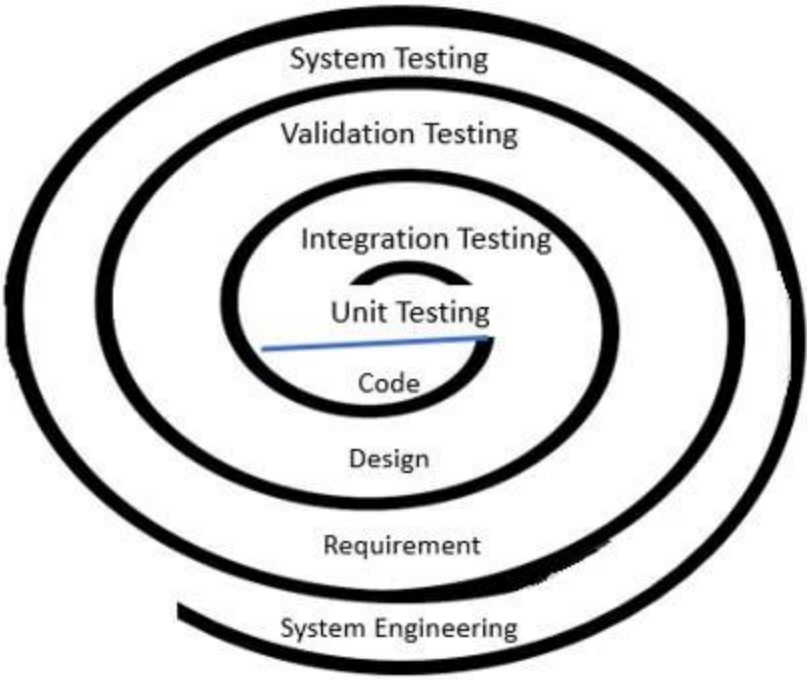
Verification	Validation
It includes checking documents, design, codes and programs.	It includes testing and validating the actual product.
Verification is the static testing.	Validation is the dynamic testing.
It does *not* include the execution of the code.	It includes the execution of the code.
Methods used in verification are reviews, walkthroughs, inspections and desk-checking.	Methods used in validation are Black Box Testing, White Box Testing and non-functional testing.
It checks whether the software conforms to specifications or not.	It checks whether the software meets the requirements and expectations of a customer or not.
It can find the bugs in the early stage of the development.	It can only find the bugs that could not be found by the verification process.

Verification	Validation
The goal of verification is application and software architecture and specification.	The goal of validation is an actual product.





(imp)Software testing strategy is an approach which incorporates planning of the steps to test the software along with the planning of time, effort & resources that will be required to test the software. Software testing strategies plans to test any software starting from the smallest component of the software and then integrating test towards the complete system.



Software Testing Strategy

Unit Testing

Unit testing focuses on testing the lowest component of the software individually which is also called unit. Unit testing involves the testing of each code segment to ensure that it functions properly. Programmers use unit testing to test pieces of codes as they develop them. Instead of testing the functionality as a whole, they perform the test on smaller units of the functionality during the development. This helps them identify any bugs or issues in the code during the early development stages. Resolving the bugs earlier can help programmers integrate the pieces of code easily, without errors. It is also convenient and economical to fix the bugs during development than to fix them after integration of the code blocks.

Integration Testing

The unit components are integrated to implement the complete software. Integration testing involves testing the design structure of the software which include modelling and software architecture. It focuses on verifying whether functions of the software are working properly or not. Integration testing is one of the main types of open box testing. It involves integrating several pieces of code to test them. The aim of this test is to see if the pieces of code work together as per the requirements. After integrating the pieces, testers run the code and identify any bugs or compatibility issues in the entire block.

Validation Testing

Validation testing focuses on the testing of software against the requirements specified by the customer.

System Testing

System testing focuses on testing the entire system as a whole and its other system elements. It tests the performance of the system.

Factors Considered to Develop Testing Strategies

1. The first and foremost thing before developing and software are gathering customer requirements. The **requirements** specified by the customer should be **measurable** so that the testing result is not ambiguous.
2. The objective of testing should be in measurable terms. Like cost required to detect and debug the error, time spends by the team to test the software.
3. While developing the testing strategy one must understand the need of the user who is going to use the software.
4. The testing strategy should implement rapid testing cycles and the generated feedback can be used in software quality control.
5. The software should be designed in a way that it is capable of diagnosing its own errors.
6. Before testing the software do the technical analysis to discover the error before testing commences.
7. The testing strategies should be improved for the better testing result of the software.

Difference Chart

Basis of Differentiation	**Black Box Testing**	**White Box Testing**
Basic	Tests the functionality of the software.	Tests all the paths in the procedural design of the software.
Schedule	Scheduled at the later stages of testing.	Scheduled at the early stages of testing.

Basis of Differentiation	Black Box Testing	White Box Testing
Testers	Testers can be the independent testers, can also be customers.	Testers should be the developers of the software.
Knowledge	Testers are not required to have the programming knowledge or even the implementation knowledge of the software.	Testers must have the knowledge of programming knowledge and must also be aware of the implementation of the software.
Test Cases	Test cases are designed by considering the functional requirement of the software.	Test cases are designed by considering the procedural design of the software.
Tests	Tests if any function is incorrect or missing, interface, database access, initiation and termination of software.	Tests if any function is incorrect or missing, interface, database access, initiation and termination of software.
Time	It is not a time-consuming process.	It consumes a considerable time.
Alternative Name	Behavioral Testing and Functional Testing.	Glass Box Testing and Structural testing.

What is White Box Testing?

White box testing is a technique of deriving test cases that ensures execution of each and every path in the program at least once during the testing of software. It is also termed as **glass box testing** or **structural testing**. The testers performing white box testing must have programming knowledge as this testing includes structural testing of software. Generally, the white box testing is performed by the developers as they are aware of the implementation of all the components of the software.

Cyclomatic complexity

Cyclomatic complexity is a software metric in open box testing that helps testers determine the complexity of a software program. It helps in identifying the number of decision points in a code. If the number of decision points is high, the complexity of the code is also high. A higher code complexity can increase the chances of errors and the time taken for the maintenance of the code.

Basis path testing

Basis path testing is a more comprehensive open box testing technique. It involves creating control graphs using flow charts or the application code. After this, the tester calculates the cyclomatic complexity of the graph, which helps in identifying the number of independent paths present in the code. The tester then designs test cases to test all the individual paths.

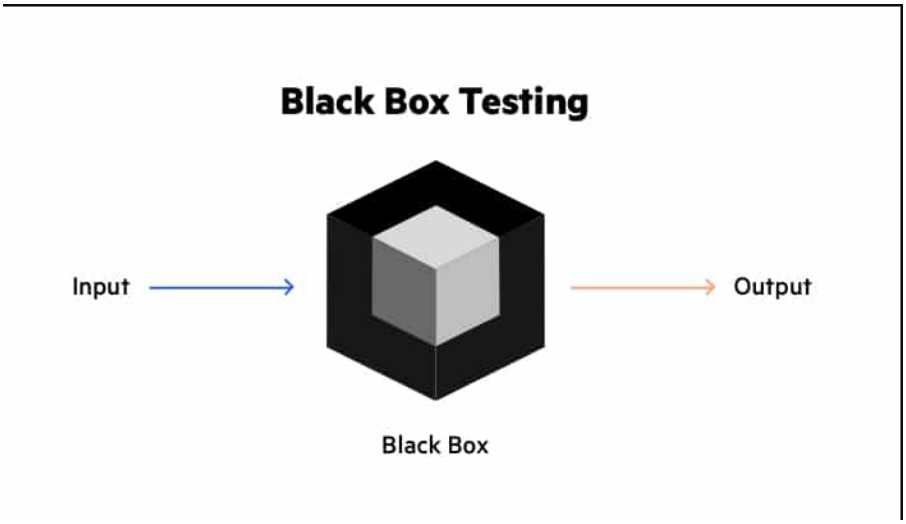
What is Black Box Testing

Black box testing involves testing a system with no prior knowledge of its internal workings. A tester provides an input, and observes the output generated by the system under test. This makes it possible to identify how the system responds to expected and unexpected user actions, its response time, usability issues and reliability issues. Black box testing is a powerful testing technique because it exercises a system end-to-end. Just like end-users "don't care" how a system is coded or architected, and expect to receive an appropriate response to their requests, a tester can simulate user activity and see if the system delivers on its promises.

Types of Black Box Testing

There are many types of Black Box Testing but the following are the prominent ones –

- **Functional testing** – This black box testing type is related to the functional requirements of a system; it is done by software testers.
- **Non-functional testing** – This type of black box testing is not related to testing of specific functionality, but non-functional requirements such as performance, scalability, usability.
- **Regression testing** – Regression Testing is done after code fixes, upgrades or any other system maintenance to check the new code has not affected the existing code.



Grey Box Testing

While white box testing assumes the tester has complete knowledge, and black box testing relies on the user's perspective with no code insight, grey box testing is a compromise. It tests applications and environments with partial knowledge of internal workings. Grey box testing is commonly used for penetration testing, end-to-end system testing, and integration testing.

Black Box Testing and Software Development Life Cycle (SDLC)

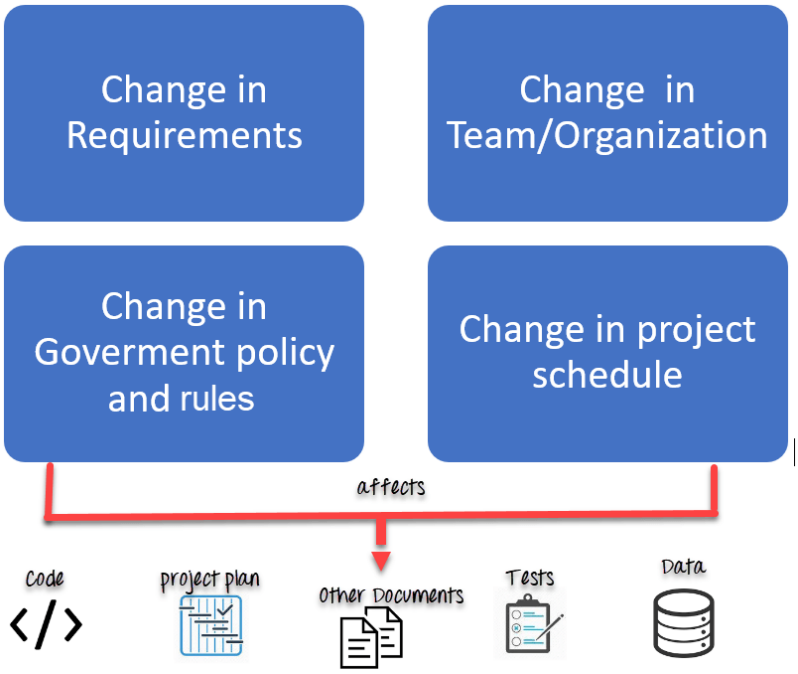
Black box testing has its own life cycle called Software Testing Life Cycle (STLC)

- **Requirement** – This is the initial stage of SDLC and in this stage, a requirement is gathered. Software testers also take part in this stage.
- **Test Planning & Analysis** – Testing Types applicable to the project are determined. A Test Plan is created which determines possible project risks and their mitigation.
- **Design** – In this stage Test cases/scripts are created on the basis of software requirement documents
- **Test Execution**– In this stage Test Cases prepared are executed. Bugs if any are fixed and re-tested.

What is Software Configuration Management?

In Software Engineering, **Software Configuration Management(SCM)** is a process to systematically manage, organize, and control the changes in the documents, codes, and other entities during the Software Development Life Cycle. The primary goal is to increase productivity with minimal mistakes. SCM is part of cross-disciplinary field of

configuration management and it can accurately determine who made which revision. Any change in the software configuration Items will affect the final product. Therefore, changes to configuration items need to be controlled and managed.



Tasks in SCM process

- Configuration Identification
- Baselines
- Change Control
- Configuration Status Accounting
- Configuration Audits and Reviews

Configuration Identification:

Configuration identification is a method of determining the scope of the software system. With the help of this step, you can manage or control something even if you don't know what it is. It is a description that contains the CSCI type (Computer Software Configuration Item), a project identifier and version information.

Baseline:

A baseline is a formally accepted version of a software configuration item. It is designated and fixed at a specific time while conducting the SCM process. It can only be changed through formal change control procedures.

Change Control:

Change control is a procedural method which ensures quality and consistency when changes are made in the configuration object. In this step, the change request is submitted to software configuration manager.

Configuration Status Accounting:

Configuration status accounting tracks each release during the SCM process. This stage involves tracking what each version has and the changes that lead to this version.

Configuration Audits and Reviews:

Software Configuration audits verify that all the software product satisfies the baseline needs. It ensures that what is built is what is delivered.

Participant of SCM process: